

THE COMPLETE

AI / ML Interview Preparation Guide

A field-tested playbook for answering with structure, intuition & confidence

Classical ML

Deep Learning

NLP & Transformers

LLMs & GenAI

46 Concepts · 4 Domains · Interview-Ready

Every concept follows one flow: how to start answering → the math intuition → pros & cons → the follow-ups interviewers actually ask.

Ravindra Sindhiya

Sr. AI / ML Engineer

 [linkedin.com/in/ravindra-sindhiya-b08609153](https://www.linkedin.com/in/ravindra-sindhiya-b08609153)

▲ Click the link above to connect with me on LinkedIn

How to use this guide

Every concept follows the same four-part structure so you can answer any interview question with a clear arc:

- **How to start answering** — a crisp 1-2 sentence opener that frames the concept before you dive into detail. Interviewers judge structure as much as content.
- **Math intuition** — the core equation and, more importantly, what it *means*. State the formula, then explain it in plain words.
- **Pros & cons** — trade-offs. Senior signals come from knowing when *not* to use something.
- **Follow-ups** — the 1-2 questions interviewers commonly ask next, with model answers.

Golden rule: structure first, depth second. Start with a one-line definition, give intuition, then the math, then a real example. Never start with the equation.

Part 1 — Classical Machine Learning

1. Linear Regression Regression

HOW TO START ANSWERING

"Linear regression models a continuous target as a weighted linear combination of input features. It's the simplest supervised baseline, and I usually reach for it first because it's fast, interpretable, and tells me whether a linear signal even exists in the data."

MATH INTUITION

Prediction: $\hat{y} = w_0 + w_1x_1 + \dots + w_nx_n = \mathbf{w}^T \mathbf{x}$

We minimise the Mean Squared Error: $J(\mathbf{w}) = (1/m) \sum (\hat{y}_i - y_i)^2$

Intuition: we draw the line (or hyperplane) that minimises the total squared vertical distance to the points. Squaring penalises large errors more and gives a smooth, convex bowl-shaped loss with a single global minimum — solvable either by the closed-form Normal Equation $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$ or by gradient descent for large data.

PROS

- Highly interpretable — each coefficient is a marginal effect.
- Fast to train, closed-form solution exists.
- Strong, hard-to-beat baseline; low variance.

CONS

- Assumes linearity; underfits complex data (high bias).
- Sensitive to outliers (squared loss).
- Assumes no multicollinearity, homoscedasticity, independent errors.

Q: Why use squared error instead of absolute error?

A: Squared error is differentiable everywhere (smooth gradients) and convex, and under the assumption of Gaussian noise it's the Maximum Likelihood Estimator. Absolute error (MAE) is more robust to outliers but its gradient is constant and non-differentiable at zero, making optimisation harder.

Q: What if features are correlated (multicollinearity)?

A: Coefficients become unstable and high-variance, and $X^T X$ becomes near-singular so the inverse is unreliable. I'd detect it with VIF, then drop/combine features or use Ridge regression, which adds an L2 penalty that stabilises the inverse.

2. Logistic Regression **Classification**

HOW TO START ANSWERING

"Despite the name, logistic regression is a classification algorithm. It models the probability of a class by passing a linear combination of features through a sigmoid, so the output is squashed to $[0,1]$. It's my go-to interpretable baseline for binary classification."

MATH INTUITION

Sigmoid: $\sigma(z) = 1 / (1 + e^{-z})$, where $z = w^T x$

Loss (Binary Cross-Entropy / Log Loss): $J = -(1/m) \sum [y \log(p) + (1-y) \log(1-p)]$

Intuition: the sigmoid turns an unbounded score into a probability. We don't use MSE here because with the sigmoid it becomes non-convex; cross-entropy is convex and punishes confident-but-wrong predictions heavily (log of a near-zero probability blows up). The decision boundary is still linear in the features.

PROS

- Outputs calibrated probabilities, not just labels.
- Interpretable; coefficients map to log-odds.
- Efficient, convex loss, hard baseline to beat.

CONS

- Linear decision boundary; can't capture complex interactions alone.
- Needs feature engineering for non-linearity.
- Sensitive to outliers and imbalanced classes.

Q: What does a coefficient actually mean here?

A: A one-unit increase in a feature changes the *log-odds* of the positive class by the

coefficient value; exponentiating it gives the odds ratio. So $e^w = 1.5$ means the odds

multiply by 1.5 per unit increase.

Q: How do you extend it to multi-class?

A: Either One-vs-Rest (one binary classifier per class) or, more cleanly, Softmax /

multinomial logistic regression, which generalises the sigmoid to a normalised distribution

over all classes with cross-entropy loss.

3. Bias–Variance Trade-off Core Concept

HOW TO START ANSWERING

"The bias-variance trade-off explains a model's generalisation error as a tug-of-war between two error sources. Bias is error from wrong assumptions (too simple), variance is error from sensitivity to the training data (too complex). The art is balancing them."

MATH INTUITION

Expected test error decomposes as: $\text{Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Noise}$

Intuition: High bias = underfitting; the model is too rigid (e.g. linear fit on a curve) and is wrong on average even with infinite data. **High variance** = overfitting; the model chases noise and changes wildly with different training samples. Increasing model complexity lowers bias but raises variance. The total-error curve is U-shaped; we want its bottom.

LOW BIAS INDICATORS

- Flexible models (deep trees, high-degree polynomials).
- Fits training data closely.

LOW VARIANCE INDICATORS

- Simple, regularised, or averaged models.
- Stable across data resamples.

Q: How do you diagnose which one you have?

A: Compare train vs. validation error. High train error + high val error = high bias

(underfit). Low train error + high val error (large gap) = high variance (overfit). Learning curves make this visual.

Q: How do you reduce variance without adding bias?

A: Get more training data, use bagging/ensembling (averaging reduces variance), or apply regularisation. More data is the cleanest fix because it shrinks variance without forcing a simpler model.

4. Regularization — L1 / L2 (Lasso / Ridge) Core Concept

HOW TO START ANSWERING

"Regularization combats overfitting by adding a penalty on the size of the model's weights to the loss function. It trades a little extra bias for a meaningful drop in variance, pushing the model toward simpler solutions that generalise better."

MATH INTUITION

L2 / Ridge: $J = \text{Loss} + \lambda \sum w_j^2$

L1 / Lasso: $J = \text{Loss} + \lambda \sum |w_j|$

Intuition: λ controls how hard we shrink weights. **L2** shrinks all weights smoothly toward (but not exactly to) zero — good when many features matter a little. **L1** has a diamond-shaped constraint whose corners touch the axes, so it drives some weights to *exactly* zero — it performs automatic feature selection and yields sparse models. Elastic Net mixes both.

PROS

- Reduces overfitting and variance.
- L1 gives feature selection / sparsity.
- L2 handles multicollinearity well.

CONS

- Adds a hyperparameter λ to tune.
- Requires feature scaling first.
- Too much λ causes underfitting.

Q: Why does L1 produce sparsity but L2 doesn't?

A: Geometrically, the L1 constraint region is a diamond with sharp corners on the axes;

the loss contours are most likely to first touch a corner, where some coefficients are

exactly zero. L2's region is a smooth circle, so the optimum rarely lands exactly on an

axis. Analytically, L1's gradient is constant ($\pm\lambda$) so it can push a weight fully to zero, while

L2's gradient ($2\lambda w$) shrinks proportionally and vanishes near zero.

Q: Why must you scale features before regularizing?

A: The penalty treats all weights equally, but a feature on a large scale gets a small

coefficient and one on a small scale gets a large coefficient. Without scaling, the penalty

unfairly punishes small-scale features. Standardise first.

5. Gradient Descent Optimization

HOW TO START ANSWERING

"Gradient descent is the workhorse optimization algorithm for minimising a loss function. It iteratively steps the parameters in the direction of the negative gradient — the direction of steepest decrease — until it reaches a minimum."

MATH INTUITION

Update rule: $w := w - \eta \nabla J(w)$

Intuition: imagine standing on a foggy hill wanting to reach the valley. The gradient points uphill (steepest ascent), so we step the opposite way. η (learning rate) is the step size: too small = painfully slow; too large = you overshoot and diverge. **Batch GD** uses all data per step (stable, slow); **SGD** uses one sample (noisy, fast, can escape local minima); **Mini-batch** uses a small batch and is the practical default.

PROS

- Scales to huge datasets (esp. mini-batch/SGD).
- General — works for any differentiable loss.
- Memory-efficient vs. closed-form on big data.

CONS

- Sensitive to learning-rate choice.
- Can get stuck in local minima / saddle points (non-convex).
- Needs feature scaling for fast convergence.

Q: Batch vs. stochastic vs. mini-batch — when each?

A: Batch for small, clean datasets where you want a smooth path. SGD when data is huge

or streaming and noise helps escape minima. Mini-batch (32–256) is the default

everywhere — it balances stable gradients with GPU-friendly vectorisation.

Q: How do you pick the learning rate?

A: Start with a search over powers of 10 (e.g. $1e-1$ to $1e-4$), watch the loss curve. Use

learning-rate schedules (decay) or adaptive optimizers like Adam. A learning-rate finder

(loss vs. increasing LR) gives a good starting band.

6. Decision Trees Tree-based

HOW TO START ANSWERING

"A decision tree splits the feature space into regions by asking a sequence of yes/no questions, choosing at each node the split that best separates the target. It's intuitive, handles non-linearity, and forms the building block of powerful ensembles."

MATH INTUITION

Splits are chosen to maximise *purity*. For classification we use Gini or Entropy:

$$\text{Gini} = 1 - \sum p_k^2 \quad \text{Entropy} = -\sum p_k \log p_k$$

Intuition: at each node we try every feature/threshold and pick the split that most reduces impurity (Information Gain = parent impurity – weighted child impurity). A pure node contains one class. For regression we minimise variance/MSE within nodes instead. The tree keeps splitting greedily until a stopping rule (max depth, min samples) kicks in.

PROS

- Very interpretable (visual rules).
- Handles non-linearity and feature interactions.
- No scaling needed; handles mixed data types.

CONS

- High variance — overfits easily.
- Unstable: small data change = different tree.
- Greedy splits aren't globally optimal; biased to high-cardinality features.

Q: Gini vs. Entropy — does it matter?

A: Rarely in practice; they produce similar trees. Gini is slightly faster (no log) and is

sklearn's default; entropy comes from information theory. Both reward pure splits.

Q: How do you stop a tree from overfitting?

A: Pre-pruning (limit max_depth, min_samples_leaf, min_samples_split) or post-pruning

(grow full, then cut weak branches via cost-complexity pruning). Better still, use an

ensemble like Random Forest.

7. Random Forest (Bagging) Ensemble

HOW TO START ANSWERING

"Random Forest is a bagging ensemble of decision trees. It trains many de-correlated trees on bootstrap samples and averages them, which dramatically reduces the variance of a single tree while keeping its low bias. It's my reliable, low-tuning default for tabular data."

MATH INTUITION

Bagging = Bootstrap Aggregating. Train each tree on a random sample (with replacement); for each split also consider only a random subset of features (typically $\sqrt{n}$).

Intuition: averaging many independent estimators reduces variance — if trees were identical, averaging wouldn't help, so the two sources of randomness (row sampling + feature sampling) *de-correlate* them. Final prediction = majority vote (classification) or mean (regression). Variance of the average $\approx \rho\sigma^2 + (1-\rho)\sigma^2/B$, so lowering correlation ρ lowers error.

PROS

- Strong accuracy with little tuning.
- Resistant to overfitting; handles noise.
- Free OOB error estimate; feature importances.

CONS

- Less interpretable than one tree.
- Large memory / slower inference (many trees).
- Can underperform boosting on structured competitions.

Q: How does it differ from boosting?

A: Bagging trains trees independently in parallel to reduce variance. Boosting trains trees sequentially, each correcting the previous one's errors, to reduce bias. RF is more robust out-of-the-box; boosting often wins accuracy but needs careful tuning and can overfit.

Q: What is OOB error?

A: Each bootstrap leaves out ~37% of rows for that tree. Those out-of-bag samples act as a built-in validation set, giving an unbiased error estimate without a separate hold-out or cross-validation.

8. Gradient Boosting (XGBoost / LightGBM) Ensemble

HOW TO START ANSWERING

"Gradient boosting builds an ensemble of weak trees sequentially, where each new tree is fit to the residual errors (the negative gradient of the loss) of the current ensemble. It's the dominant method for tabular data and wins most Kaggle competitions."

MATH INTUITION

At step m : $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$, where h_m is fit to the negative gradient of the loss.

Intuition: instead of averaging independent trees, we add trees one at a time, each one a small correction in the direction that most reduces the remaining error — this is gradient descent in function space. η (learning rate / shrinkage) keeps each step small so we don't overcorrect. XGBoost adds L1/L2 regularisation and a second-order (Newton) approximation for better, faster fits.

PROS

- State-of-the-art accuracy on tabular data.
- Handles mixed features, missing values (XGBoost).
- Built-in regularisation; flexible loss functions.

CONS

- More hyperparameters; sensitive to tuning.
- Sequential — slower to train than RF.
- Can overfit if learning rate / depth not controlled.

Q: Why does a small learning rate help?

A: Shrinkage scales each tree's contribution, so the model improves in many small steps rather than a few big ones. This regularises and improves generalisation — but needs more trees (`n_estimators`), so there's a learning-rate vs. number-of-trees trade-off, usually tuned with early stopping.

Q: XGBoost vs. LightGBM?

A: LightGBM grows trees leaf-wise (best-first) and uses histogram binning, making it much faster on large data, but it can overfit small datasets. XGBoost grows level-wise and is more robust. Both are excellent; LightGBM for scale, XGBoost as a safe default.

9. Support Vector Machines (SVM) Classification

HOW TO START ANSWERING

"An SVM finds the decision boundary that maximises the margin — the distance between the boundary and the nearest points of each class (the support vectors). Maximising this margin gives strong generalisation, and the kernel trick lets it handle non-linear boundaries."

MATH INTUITION

Maximise the margin $2 / ||w||$, i.e. minimise $\frac{1}{2}||w||^2$ subject to $y_i(w^T x_i + b) \geq 1$

Intuition: among all separating lines, we pick the widest "street" between the classes;

only the closest points (support vectors) define it. A soft margin (parameter C) allows some

misclassifications to handle overlap — small C = wider margin, more tolerance; large C =

fewer errors, risk of overfit. The **kernel trick** (e.g. RBF) implicitly maps data to higher

dimensions where it's linearly separable, without computing those coordinates.

PROS

- Effective in high-dimensional spaces.
- Kernel trick handles non-linearity elegantly.
- Robust via margin maximisation; memory-efficient (only SVs).

CONS

- Poor scaling to very large datasets ($O(n^2-n^3)$).
- Needs careful kernel & C/gamma tuning.
- No native probabilities; sensitive to scaling.

Q: What is the kernel trick, intuitively?

A: It computes the dot product between points *as if* they were mapped to a higher-

dimensional space, using a kernel function, without ever materialising those coordinates.

So we get non-linear boundaries at the cost of a cheap kernel evaluation. RBF is the

common default.

Q: What do C and gamma control in an RBF SVM?

A: C is the regularisation / margin-softness trade-off (high C = fit training data tightly).

Gamma is the reach of a single training point (high gamma = tight, wiggly boundary that

can overfit; low gamma = smooth). They're tuned together via grid/random search.

10. K-Nearest Neighbors (KNN) Instance-based

HOW TO START ANSWERING

"KNN is a lazy, instance-based algorithm: to classify a point, it looks at the k closest training examples and takes a majority vote (or averages for regression). There's no training phase — the data *is* the model."

MATH INTUITION

Distance (Euclidean): $d(x, x') = \sqrt{(\sum (x_j - x'_j)^2)}$

Intuition: similar inputs have similar outputs, so we predict from the local neighbourhood.

k controls the bias-variance balance: small k = flexible, noisy boundary (low bias, high variance); large k = smooth boundary (high bias, low variance). Because it relies on distance, feature scaling is essential, and it suffers in high dimensions (curse of dimensionality — distances become meaningless).

PROS

- Dead simple, no training cost.
- Naturally handles multi-class & non-linear boundaries.
- Few assumptions about data.

CONS

- Slow, memory-heavy inference (search all points).
- Curse of dimensionality; needs scaling.
- Sensitive to irrelevant features and k.

Q: How do you choose k?

A: Cross-validate over a range; use an odd k for binary classification to avoid ties. A

common rule of thumb is $k \approx \sqrt{n}$ as a starting point, then refine by validation error.

Q: Why does KNN struggle in high dimensions?

A: As dimensions grow, all points become roughly equidistant, so "nearest" loses meaning and neighbourhoods are no longer local. Mitigate with dimensionality reduction (PCA) or feature selection before applying KNN.

11. Naive Bayes Probabilistic

HOW TO START ANSWERING

"Naive Bayes is a probabilistic classifier built on Bayes' theorem with a 'naive' assumption that features are conditionally independent given the class. Despite that strong assumption, it's fast, scales beautifully, and is a strong baseline for text classification."

MATH INTUITION

Bayes: $P(y | x) \propto P(y) \cdot \prod_j P(x_j | y)$

Intuition: we want the most probable class given the features. By Bayes' rule that's proportional to the prior times the likelihood. The naive independence assumption lets us multiply per-feature likelihoods instead of modelling their joint distribution — turning a hard problem into simple counting. We pick the class with the highest posterior (MAP). Variants: Multinomial (counts/text), Gaussian (continuous), Bernoulli (binary).

PROS

- Extremely fast; great for high-dimensional text.
- Works well with little training data.
- Naturally handles multi-class; gives probabilities.

CONS

- Independence assumption rarely true.
- Probabilities are poorly calibrated.
- "Zero-frequency" problem needs smoothing.

Q: What is Laplace (additive) smoothing and why?

A: If a feature value never appeared with a class in training, its likelihood is zero, which zeros the whole product. Laplace smoothing adds a small count (alpha, usually 1) to every count so no probability is ever exactly zero.

Q: Why does it work so well for text despite the naive assumption?

A: Even when independence is violated, the *ranking* of class posteriors is often still correct, which is all we need for classification. Text is also high-dimensional and sparse, where the simple counting model is robust and cheap.

12. K-Means Clustering Unsupervised

HOW TO START ANSWERING

"K-Means is an unsupervised algorithm that partitions data into k clusters by iteratively assigning each point to its nearest centroid and then moving each centroid to the mean of its assigned points. It's the go-to for fast, simple clustering when you roughly know k ."

MATH INTUITION

Minimise within-cluster sum of squares (inertia): $J = \sum_k \sum_{x \in C_k} ||x - \mu_k||^2$

Intuition: it's an alternating optimisation (a form of EM). Step 1 (assign): each point joins the closest centroid. Step 2 (update): each centroid moves to the average of its members. Repeat until assignments stop changing. Each step can only lower J , so it converges — but only to a *local* optimum, which is why we use smart initialisation (k-means++) and multiple restarts.

PROS

- Fast and scalable (linear in n).
- Simple, easy to interpret.
- Works well on globular, well-separated clusters.

CONS

- Must pre-specify k.
- Assumes spherical, equal-size clusters; sensitive to scale & outliers.
- Converges to local optima; depends on initialisation.

Q: How do you choose k?

A: The Elbow method (plot inertia vs. k, look for the bend) or the Silhouette score

(measures cohesion vs. separation, higher is better). Domain knowledge often matters more than either.

Q: When does K-Means fail, and what's the alternative?

A: It fails on non-spherical, varying-density, or nested clusters because it only draws linear (Voronoi) boundaries. Use DBSCAN (density-based, finds arbitrary shapes and outliers, no k needed) or Gaussian Mixture Models (soft, elliptical clusters).

13. Principal Component Analysis (PCA) Dimensionality Reduction

HOW TO START ANSWERING

"PCA is an unsupervised dimensionality-reduction technique. It finds new orthogonal axes — the principal components — ordered by how much variance they capture, letting us compress the data to fewer dimensions while keeping most of the information."

MATH INTUITION

Find eigenvectors of the covariance matrix $\Sigma = (1/m) X^T X$; the top-k eigenvectors (largest eigenvalues) are the principal components.

Intuition: directions where the data varies most carry the most information. PCA rotates the axes to align with these directions of maximum variance; the first PC captures the most spread, the second (orthogonal to it) the next most, and so on. Keeping the top components that explain, say, 95% of variance discards mostly noise. Equivalent to SVD of the centred data matrix.

PROS

- Reduces dimensionality, speeds up models, fights overfitting.
- Removes correlated features; aids visualisation (2D/3D).
- De-noises data.

CONS

- Components are linear combinations — hard to interpret.
- Only captures linear structure.
- Sensitive to scaling; variance $\neq$ predictive importance.

Q: Should you scale before PCA?

A: Yes — PCA chases variance, so a feature with a large numeric range would dominate the components purely due to its units. Standardise to zero mean and unit variance first (unless features are already on the same scale).

Q: PCA vs. feature selection — difference?

A: Feature selection keeps a subset of the original, interpretable features. PCA creates new features (combinations of all originals), so you lose interpretability but can capture variance more compactly. Also, the top variance directions aren't guaranteed to be the most predictive for a supervised task — that's a known caveat.

14. Evaluation Metrics (Classification) Evaluation

HOW TO START ANSWERING

"I always pick the metric that matches the business cost, not just accuracy. I start from the confusion matrix and derive precision, recall, F1, and ROC-AUC, then choose based on class balance and which error — false positives or false negatives — is costlier."

MATH INTUITION

Precision = $TP / (TP + FP)$ — of those I flagged positive, how many were right?

Recall = $TP / (TP + FN)$ — of all true positives, how many did I catch?

F1 = $2 \cdot (P \cdot R) / (P + R)$ — harmonic mean, balances both.

Intuition: Precision matters when false positives are costly (spam filter flagging real mail).

Recall matters when false negatives are costly (cancer screening, fraud). F1 balances them on imbalanced data. ROC-AUC measures ranking quality across all thresholds; PR-AUC is better when positives are rare.

USE RECALL / PR-AUC WHEN

- Classes are imbalanced.
- Missing a positive is expensive (fraud, disease).

USE PRECISION / ACCURACY WHEN

- False alarms are expensive.
- Classes are balanced (accuracy is fine).

Q: Why is accuracy misleading on imbalanced data?

A: With 99% negatives, a model that always predicts "negative" scores 99% accuracy but is useless — it catches zero positives. Precision/recall/F1 or AUC expose this; accuracy hides it.

Q: ROC-AUC vs. PR-AUC?

A: ROC-AUC plots TPR vs. FPR and is threshold-independent, but it can look optimistic

when negatives massively outnumber positives. PR-AUC (precision vs. recall) focuses on

the positive class and is more informative for rare-event problems.

15. Cross-Validation & Overfitting Validation

HOW TO START ANSWERING

"Cross-validation gives a more reliable estimate of generalisation than a single train/test split by rotating the validation set across the data. I use k-fold CV both to estimate performance honestly and to tune hyperparameters without leaking the test set."

MATH INTUITION

k-Fold: split data into k folds; train on $k-1$, validate on the held-out fold, rotate k times, average the scores.

Intuition: a single split is a lucky/unlucky sample — the score has high variance.

Averaging k splits uses every point for both training and validation, giving a lower-variance estimate. Common k is 5 or 10. For imbalanced classes use **stratified** k-fold (preserves class ratios); for time series use forward-chaining (never train on the future).

PROS

- Reliable, low-variance performance estimate.
- Uses all data; great for small datasets.
- Enables principled hyperparameter tuning.

CONS

- $k \times$ more compute.
- Wrong CV scheme leaks data (time series, groups).
- Not a substitute for a final untouched test set.

Q: How do you cross-validate time-series data?

A: Never shuffle — that leaks the future into training. Use rolling/expanding-window

(forward chaining): train on [1..t], validate on t+1, then expand. This respects temporal order.

Q: What is data leakage and how do you prevent it in CV?

A: Leakage is when information from the validation/test set sneaks into training (e.g.

scaling or imputing using the full dataset's statistics). Fix it by fitting all preprocessing

inside the CV loop — use a Pipeline so transforms are learned only on the training fold.

16. Handling Imbalanced Data Practical

HOW TO START ANSWERING

"With imbalanced classes I attack it on three fronts: the right metric (not accuracy), data-level resampling, and algorithm-level class weights. I decide based on how rare the minority class is and the cost of each error type."

MATH INTUITION

SMOTE creates synthetic minority points by interpolating between a sample and its k nearest minority neighbours: $x_{\text{new}} = x_i + \delta(x_{\text{nn}} - x_i)$, $\delta \in [0,1]$

Intuition: the model sees too few positives to learn them, so it ignores them. We either oversample the minority (SMOTE adds plausible new points instead of mere copies, reducing overfit), undersample the majority, or tell the loss to weight minority errors more heavily (class_weight). Threshold tuning on the probability output is also powerful and free.

APPROACHES

- Resampling: SMOTE / oversample, undersample.
- Class weights in the loss function.
- Threshold moving; anomaly-detection framing.

WATCH-OUTS

- Oversampling can overfit; SMOTE before split = leakage.
- Undersampling discards information.
- Always evaluate with PR-AUC / recall, not accuracy.

Q: SMOTE vs. class weights — which do you prefer?

A: I try class weights first — it's simple, leak-free, and changes only the loss. SMOTE can

help when the minority is extremely rare and the model needs richer examples, but it

must be applied *only on the training fold* to avoid leakage.

Q: Where do people accidentally leak with SMOTE?

A: Applying SMOTE to the whole dataset before train/test splitting — synthetic points

derived from test rows leak in, inflating scores. Always split first, then resample inside the

pipeline on training data only.

17. Feature Scaling & Encoding Preprocessing

HOW TO START ANSWERING

"Good preprocessing often beats a fancier model. I scale numeric features for any distance- or gradient-based algorithm, and encode categoricals appropriately — the right choice depends on the model family and cardinality."

MATH INTUITION

Standardisation: $x' = (x - \mu) / \sigma$ (zero mean, unit variance).

Min-Max: $x' = (x - \min) / (\max - \min)$ (scales to [0,1]).

Intuition: algorithms that use distances (KNN, K-Means, SVM) or gradients (linear/logistic, neural nets) treat large-magnitude features as more important purely by scale. Standardisation fixes this and speeds gradient descent convergence (round, not elongated, loss surface). Tree-based models are scale-invariant and don't need it. For categoricals: one-hot for low cardinality, target/ordinal encoding for high cardinality.

NEED SCALING

- KNN, K-Means, SVM, PCA.
- Linear/logistic regression, neural networks.

DON'T NEED SCALING

- Decision trees, Random Forest, Gradient Boosting.
- Rule-based models.

Q: Standardisation vs. normalisation — when each?

A: Standardisation (z-score) when data is roughly Gaussian or for algorithms assuming

centred data (PCA, linear models); it's robust to mild outliers. Min-max when you need a

bounded range (e.g. image pixels, some neural nets) but it's sensitive to outliers.

RobustScaler (uses median/IQR) when heavy outliers exist.

Q: How do you encode a high-cardinality categorical like zip code?

A: One-hot would explode dimensionality. Use target/mean encoding (replace category

with the mean target, with smoothing and inside CV to avoid leakage), frequency

encoding, or learned embeddings in a neural net.

Part 2 — Deep Learning

18. Neural Networks & the Perceptron Foundations

HOW TO START ANSWERING

"A neural network is a stack of layers of simple units (neurons), each computing a weighted sum of its inputs followed by a non-linear activation. Stacking these layers lets the network learn hierarchical, non-linear feature representations directly from data."

MATH INTUITION

One neuron: $a = \phi(w^T x + b)$, a layer applies this in parallel; layers compose: $h = \phi_2(W_2 \phi_1(W_1 x))$

Intuition: each neuron is essentially a tiny logistic regression. Linear layers alone collapse to one linear map, so the **non-linear activation** is what gives the network its power — the Universal Approximation Theorem says a network with one hidden layer can approximate any continuous function given enough neurons. Depth makes this efficient: early layers learn simple features (edges), later layers compose them (shapes, objects).

PROS

- Learns features automatically (no manual engineering).
- Universal approximators; excel on images, text, audio.
- Scale with data and compute.

CONS

- Data- and compute-hungry.
- Black-box; hard to interpret.
- Many hyperparameters; prone to overfitting.

Q: Why do we need non-linear activations?

A: Without them, composing linear layers is still just one linear function — no matter the depth, the network could only learn linear boundaries. Non-linearity is what lets depth model complex functions.

Q: What does the Universal Approximation Theorem really say?

A: That a wide-enough single hidden layer can approximate any continuous function on a bounded domain — but it says nothing about *learnability* or efficiency. In practice deep narrow networks learn far more parameter-efficiently than shallow wide ones.

19. Backpropagation Foundations

HOW TO START ANSWERING

"Backpropagation is the algorithm that trains neural networks. It efficiently computes the gradient of the loss with respect to every weight by applying the chain rule backward through the network, so we know how to nudge each weight to reduce the loss."

MATH INTUITION

Chain rule across a layer: $\partial L / \partial w = (\partial L / \partial a) \cdot (\partial a / \partial z) \cdot (\partial z / \partial w)$

Intuition: a forward pass computes the prediction and loss. Then we ask, "how much did each weight contribute to the error?" Starting at the output and moving backward, we multiply local derivatives (chain rule), reusing already-computed gradients from later layers — that reuse is what makes it efficient ($O(\text{network size})$ instead of recomputing everything per weight). The result feeds gradient descent.

PROS

- Efficient exact gradients for millions of params.
- Foundation of all deep learning training.
- Automatic via autodiff frameworks.

CONS

- Suffers vanishing/exploding gradients in deep nets.
- Requires differentiable operations.
- Memory cost: must cache activations for the backward pass.

Q: Why compute gradients backward rather than forward?

A: With one scalar loss and many parameters, reverse-mode (backprop) gets all gradients in a single backward pass; forward-mode would need one pass per parameter. Reverse mode is far cheaper when outputs $\ll$ inputs.

Q: What gets cached during the forward pass and why?

A: The intermediate activations, because the backward pass needs them to compute local derivatives. That's why training memory scales with batch size and depth — and why techniques like gradient checkpointing trade compute for memory.

20. Activation Functions Architecture

HOW TO START ANSWERING

"Activation functions inject non-linearity so the network can learn complex patterns. The choice matters for gradient flow — I default to ReLU (or its variants) in hidden layers and pick the output activation based on the task: sigmoid for binary, softmax for multi-class, linear for regression."

MATH INTUITION

$\text{ReLU}(z) = \max(0, z)$ $\text{Sigmoid} = 1/(1+e^{-z})$ $\text{Tanh} = (e^z - e^{-z})/(e^z + e^{-z})$

Intuition: Sigmoid/Tanh squash inputs but saturate — for large $|z|$ their gradient $\rightarrow 0$, causing vanishing gradients in deep nets. ReLU keeps a gradient of 1 for positive inputs, so it trains deep nets fast and is sparse (zeros out negatives), but it can "die" (stuck at 0). Leaky ReLU / GELU / ELU fix the dying problem by allowing a small negative slope.

RELU PROS

- No saturation for $z > 0$ — fast training.
- Cheap to compute; induces sparsity.

RELU CONS

- "Dying ReLU": neurons stuck outputting 0.

- Unbounded output; not zero-centred.

Q: What is the dying ReLU problem and how do you fix it?

A: If a neuron's weights push its input always negative, ReLU outputs 0 with zero

gradient, so it never updates again — effectively dead. Fixes: Leaky ReLU / Parametric

ReLU (small negative slope), ELU/GELU, careful initialisation, and lower learning rates.

Q: Why softmax for the output but ReLU for hidden layers?

A: Hidden layers need cheap, non-saturating non-linearity for gradient flow (ReLU). The

output for multi-class needs a probability distribution that sums to 1 — that's exactly

what softmax produces, pairing naturally with cross-entropy loss.

21. Loss Functions Training

HOW TO START ANSWERING

"The loss function defines what 'good' means — it's the objective the network minimises. I match it to the task: MSE/MAE for regression, cross-entropy for classification, and specialised losses (contrastive, focal, etc.) for specific problems."

MATH INTUITION

MSE (regression): $(1/m)\sum(\hat{y}-y)^2$

Cross-Entropy (classification): $-\sum y \log(\hat{y})$

Intuition: the loss must be differentiable and align with the goal. Cross-entropy comes from maximum likelihood — it heavily penalises confident wrong predictions (the log term explodes as the predicted probability of the true class $\rightarrow 0$), giving strong gradients early in training. MSE assumes Gaussian errors; MAE is more outlier-robust. Focal loss down-weights easy examples for imbalanced detection.

CROSS-ENTROPY PROS

- Strong gradients; fast convergence with softmax.
- Probabilistically grounded (MLE).

MSE-FOR-CLASSIFICATION CONS

- Weak gradients when paired with sigmoid.
- Non-convex; slower, can stall.

Q: Why cross-entropy instead of MSE for classification?

A: With a sigmoid/softmax output, MSE's gradient nearly vanishes when predictions are

very wrong (saturation), so learning stalls. Cross-entropy's gradient stays large for

confident mistakes, giving faster, more stable training. It's also the proper MLE objective

for categorical targets.

Q: When would you use a custom loss like focal loss?

A: In heavily imbalanced problems (e.g. dense object detection) where easy negatives

swamp the signal. Focal loss multiplies cross-entropy by $(1-p)^{\gamma}$, shrinking the

contribution of easy, well-classified examples so the model focuses on the hard minority.

22. Optimizers (SGD, Momentum, Adam) Training

HOW TO START ANSWERING

"Optimizers decide how to use gradients to update weights. They range from plain SGD to adaptive methods like Adam. I usually start with Adam for fast convergence, and switch to SGD with momentum when I want the best final generalisation."

MATH INTUITION

Momentum: $v := \beta v + \nabla J$; $w := w - \eta v$

Adam combines momentum (1st moment) + RMSProp (2nd moment): adapts a per-parameter learning rate.

Intuition: plain SGD takes equal steps and zig-zags in ravines. **Momentum** accumulates a velocity, like a ball rolling downhill — it smooths oscillations and accelerates along consistent directions. **Adam** additionally scales each parameter's step by a running estimate of its gradient magnitude, so rarely-updated and steep parameters get appropriately sized steps. It's robust and needs little tuning.

ADAM PROS

- Fast convergence; adaptive per-parameter LR.
- Robust default; little tuning.
- Handles sparse gradients well.

ADAM CONS

- Can generalise slightly worse than SGD+momentum.
- More memory (stores moments).
- Can converge to sharp minima; may need weight-decay fix (AdamW).

Q: Adam vs. SGD — which and why?

A: Adam for fast prototyping, NLP/transformers, and sparse gradients. SGD with

momentum (plus an LR schedule) often reaches a flatter minimum that generalises

better, which is why many vision SOTA models still train with it. A common recipe is Adam

early, SGD to fine-tune.

Q: What problem does AdamW solve?

A: In vanilla Adam, L2 regularisation gets entangled with the adaptive scaling, weakening it. AdamW *decouples* weight decay from the gradient update, applying it directly to the weights — giving proper regularisation and better generalisation. It's now the default for transformers.

23. Vanishing & Exploding Gradients Core Problem

HOW TO START ANSWERING

"These are the classic problems that make deep networks hard to train. As gradients propagate backward through many layers, they get multiplied repeatedly — if the factors are <1 they shrink toward zero (vanishing), if >1 they blow up (exploding), so early layers stop learning or training diverges."

MATH INTUITION

Backprop multiplies many Jacobians: $\partial L / \partial w_1 \propto \prod (W \cdot \phi')$

Intuition: a product of many numbers either decays to 0 or explodes depending on whether each term is below or above 1. Saturating activations (sigmoid/tanh) have small derivatives, compounding the vanishing. Solutions tackle each factor: ReLU (derivative 1), careful initialisation (He/Xavier keeps variance stable), batch norm, residual/skip connections (provide a gradient highway), and gradient clipping for explosions.

FIXES FOR VANISHING

- ReLU activations; He initialisation.
- Batch norm; residual/skip connections (ResNet).
- LSTM/GRU gates for sequences.

FIXES FOR EXPLODING

- Gradient clipping (by norm/value).
- Proper initialisation; lower learning rate.
- Weight regularisation.

Q: How do residual connections help?

A: A skip connection computes $y = F(x) + x$, so the gradient has a direct path (the $+x$ term contributes a gradient of 1) that bypasses the layers. This lets gradients flow to early layers undiminished, enabling networks hundreds of layers deep (ResNet).

Q: How do you detect which one you have?

A: Monitor gradient norms per layer. Vanishing = early-layer gradients near zero and stalled loss; exploding = NaNs or wildly oscillating loss. TensorBoard histograms of gradients/weights make this visible.

24. Weight Initialization Training

HOW TO START ANSWERING

"How we initialise weights strongly affects whether a deep net trains at all. The goal is to keep the variance of activations and gradients stable across layers, so I use He initialisation with ReLU and Xavier/Glorot with tanh/sigmoid."

MATH INTUITION

Xavier: $\text{Var}(w) = 1 / n_{\text{in}}$ (or $2 / (n_{\text{in}} + n_{\text{out}})$) He: $\text{Var}(w) = 2 / n_{\text{in}}$

Intuition: if weights are too large, activations explode; too small, they vanish — both kill learning. We scale the initial variance by the number of incoming connections so the signal keeps roughly unit variance layer to layer. He uses a factor of 2 because ReLU zeros out half the inputs, halving variance. Never initialise all weights to zero — then all neurons compute the same thing and learn identically (symmetry).

GOOD INIT GIVES

- Stable signal/gradient variance.
- Faster, more reliable convergence.
- Breaks neuron symmetry.

BAD INIT CAUSES

- Vanishing/exploding from layer one.
- Dead neurons or stalled training.
- All-zero = no learning.

Q: Why can't you initialise all weights to zero?

A: Every neuron in a layer would receive identical gradients and update identically forever

— the network can't break symmetry, so it behaves like a single neuron. Random init breaks this.

Q: He vs. Xavier — when each?

A: He init for ReLU-family activations (accounts for the half that ReLU zeros).

Xavier/Glorot for tanh/sigmoid, which are symmetric around zero. With batch/layer norm the network is more forgiving, but matching init to activation still helps early training.

25. Batch Normalization Regularization

HOW TO START ANSWERING

"Batch normalization normalises each layer's inputs per mini-batch to zero mean and unit variance, then rescales them with two learned parameters. It stabilises and accelerates training and has a mild regularising effect."

MATH INTUITION

$$\hat{x} = (x - \mu_{\text{batch}}) / \sqrt{(\sigma^2_{\text{batch}} + \epsilon)}; \quad y = \gamma \hat{x} + \beta$$

Intuition: as earlier layers update, the distribution of inputs to later layers keeps shifting ("internal covariate shift"), forcing them to constantly re-adapt. BatchNorm re-centres and re-scales each batch to a stable distribution, smoothing the loss landscape so we can use higher learning rates. The learned γ, β let the network undo the normalisation if needed. At inference, running averages of mean/variance are used.

PROS

- Faster training; allows higher learning rates.
- Reduces sensitivity to initialisation.
- Slight regularisation (batch noise).

CONS

- Depends on batch size; poor for tiny batches.
- Awkward for RNNs / variable-length sequences.
- Different train vs. inference behaviour (a bug source).

Q: Why does BatchNorm behave differently at train vs. test time?

A: During training it uses the current mini-batch's mean/variance; at inference there's no batch, so it uses running averages accumulated during training. Forgetting to switch to eval mode is a classic bug that corrupts predictions.

Q: BatchNorm vs. LayerNorm?

A: BatchNorm normalises across the batch dimension (great for CNNs, batch-size-dependent). LayerNorm normalises across features within a single sample, so it's independent of batch size — that's why transformers and RNNs use LayerNorm.

26. Dropout & Regularization in DL Regularization

HOW TO START ANSWERING

"Dropout is a regularisation technique that randomly switches off a fraction of neurons during each training step, forcing the network not to rely on any single neuron. It's like training an ensemble of sub-networks that share weights."

MATH INTUITION

During training, each neuron is kept with probability p ; activations are scaled by $1/p$ (inverted dropout) so the expected value is unchanged.

Intuition: by randomly dropping neurons, the network can't co-adapt — no feature can depend on a specific partner always being present, so it learns redundant, robust representations. It approximates averaging many thinned networks (an implicit ensemble). At test time, dropout is off and all neurons are used. Typical rates: 0.2–0.5. Combine with L2/weight decay, data augmentation, and early stopping.

PROS

- Strong, cheap regulariser against overfitting.
- Acts like an ensemble; improves robustness.
- Easy to add to any layer.

CONS

- Slows convergence (noisier training).
- Redundant with BatchNorm; can interact badly.
- Wrong rate hurts; off at inference (easy to misconfigure).

Q: Why scale activations by $1/p$ during training?

A: So the expected sum of inputs to the next layer is the same with and without dropout.

This "inverted dropout" means no scaling is needed at test time — inference stays simple and matches the training expectation.

Q: Should you use Dropout and BatchNorm together?

A: Often not in the same spot — BatchNorm already regularises, and dropout's

randomness can disturb its batch statistics. A common pattern is BatchNorm in conv

layers and dropout in the dense head, or just one of the two. Test empirically.

27. Convolutional Neural Networks (CNNs) Architecture

HOW TO START ANSWERING

"CNNs are the standard architecture for grid-like data such as images. Instead of connecting every pixel to every neuron, they slide small learnable filters across the input to detect local patterns, sharing weights to exploit the fact that a feature is useful anywhere in the image."

MATH INTUITION

Convolution: a filter (kernel) slides over the input computing dot products — $(I * K)(i,j) = \sum_l \sum_n I(i+m, j+n) K(m,n)$

Intuition: three ideas make CNNs efficient. **Local receptive fields** — nearby pixels are related, so filters look at small patches. **Parameter sharing** — the same filter scans the whole image, so far fewer weights and translation invariance. **Hierarchy** — early filters detect edges, deeper ones combine them into textures, parts, then objects. Pooling downsamples for invariance and efficiency.

PROS

- Parameter-efficient via weight sharing.
- Translation invariance; captures spatial hierarchy.
- State-of-the-art on vision tasks.

CONS

- Data- and compute-hungry.
- Not naturally rotation/scale invariant.
- Large receptive fields need depth; less interpretable.

Q: What does pooling do and why?

A: Pooling (e.g. max pooling) downsamples feature maps, reducing spatial size and computation while giving small translation invariance — the exact position of a feature matters less than its presence. Many modern nets replace it with strided convolutions.

Q: Why are CNNs more efficient than fully-connected nets for images?

A: A dense layer on a 224×224 image would need billions of weights. CNNs share a small filter across all positions, cutting parameters massively and building in the prior that visual features are local and translation-invariant — which also reduces overfitting.

28. RNNs, LSTMs & GRUs Sequence Models

HOW TO START ANSWERING

"RNNs process sequences by maintaining a hidden state that carries information from previous steps — they have memory. Vanilla RNNs struggle with long-range dependencies due to vanishing gradients, which LSTMs and GRUs solve with gating mechanisms."

MATH INTUITION

RNN: $h_t = \phi(W_h h_{t-1} + W_x x_t)$ — same weights applied at every step.

Intuition: the hidden state is a running summary of the sequence so far. But backpropagating through many time steps multiplies the same weight repeatedly → gradients vanish/explode, so the network forgets distant context. **LSTMs** add a cell state plus input/forget/output gates that learn what to keep, erase, and expose — the additive cell update lets gradients flow far back. **GRUs** are a lighter 2-gate version with similar performance.

PROS

- Handle variable-length sequences.
- LSTM/GRU capture long-range dependencies.
- Parameter-efficient (shared across time).

CONS

- Sequential — can't parallelise over time (slow).
- Still struggle with very long contexts.
- Largely superseded by Transformers.

Q: How does an LSTM solve vanishing gradients?

A: The cell state is updated additively and gated, creating a near-constant-error

"conveyor belt" through time. The forget gate can hold information unchanged across

many steps, so gradients don't shrink multiplicatively the way they do in a vanilla RNN.

Q: LSTM vs. GRU — which?

A: GRU has fewer gates/parameters, trains faster, and often matches LSTM on smaller

datasets. LSTM's separate cell state can help on longer or more complex sequences. I'd

try GRU first for speed, LSTM if I need more capacity — though for most sequence tasks

today I'd reach for a Transformer.

29. Transfer Learning & Fine-Tuning Practical

HOW TO START ANSWERING

"Transfer learning reuses a model pre-trained on a large dataset as a starting point for a related task with less data. Instead of training from scratch, I leverage features the model already learned, which saves compute and works remarkably well on small datasets."

MATH INTUITION

Strategy depends on data size & similarity: **feature extraction** (freeze the backbone, train a new head) vs. **fine-tuning** (unfreeze some/all layers with a small learning rate).

Intuition: early layers learn generic features (edges, textures, basic language) that transfer across tasks; later layers are task-specific. So we keep the general layers and retrain the specific ones. With little data, freeze more to avoid overfitting; with more data or a very different domain, fine-tune deeper. Use a low learning rate so we don't destroy the pre-trained weights.

PROS

- Works with small datasets; less compute.
- Faster convergence, higher accuracy.
- Reuses billions of dollars of pre-training.

CONS

- Negative transfer if domains differ too much.
- Large pre-trained models can be heavy to deploy.
- Risk of catastrophic forgetting when fine-tuning.

Q: When freeze vs. fine-tune?

A: Small data + similar domain → freeze the backbone, train only the head. Lots of data

and/or a different domain → fine-tune more layers. A common recipe: train the new head

first, then unfreeze top layers and fine-tune with a very low learning rate.

Q: What is catastrophic forgetting?

A: When fine-tuning overwrites the useful pre-trained knowledge because the new

gradients are too large. Mitigate with low learning rates, gradual unfreezing,

discriminative learning rates (smaller for earlier layers), or parameter-efficient methods

like LoRA.

30. Hyperparameter Tuning & Learning Rate Practical

HOW TO START ANSWERING

"Hyperparameters are the settings we choose before training — learning rate, batch size, architecture, regularisation. The learning rate is the single most important one. I tune systematically with a validation set, starting from sensible defaults and using random or Bayesian search rather than blind grid search."

MATH INTUITION

The learning rate η scales every gradient step: $w := w - \eta \nabla J$

Intuition: too high → the loss oscillates or diverges; too low → painfully slow, may stall in a poor minimum. Schedules help: warmup (start small to stabilise), then decay (step, cosine) to settle into a minimum. Random search beats grid search because only a few hyperparameters usually matter, and random sampling explores those dimensions better per trial. Bayesian optimisation models the validation surface to pick promising points.

SEARCH METHODS

- Random search: efficient, strong default.
- Bayesian opt (Optuna): sample-efficient.
- LR schedules: warmup + cosine decay.

PITFALLS

- Grid search scales exponentially.
- Tuning on the test set = leakage.
- Compute-expensive; needs early stopping.

Q: Why is random search often better than grid search?

A: With a fixed budget, grid search wastes trials on hyperparameters that don't matter, sampling each important one at only a few values. Random search samples every dimension at a new value each trial, so it covers the important dimensions far more densely — proven to find better configs per unit compute.

Q: How do you find a good learning rate quickly?

A: An LR range test — train for a few hundred steps while exponentially increasing the LR and plot loss vs. LR. Pick a value about an order of magnitude below where the loss starts diverging. Pair it with warmup and cosine/step decay.

31. Embeddings Representation

HOW TO START ANSWERING

"An embedding is a learned, dense, low-dimensional vector representation of a discrete object — a word, user, product, or category — where similar items end up close together in the vector space. They replace sparse one-hot encodings with meaningful, learnable representations."

MATH INTUITION

An embedding layer is a lookup table $E \in \mathbb{R}^{V \times d}$: each of V items maps to a d -dimensional vector, learned by gradient descent.

Intuition: one-hot vectors are huge, sparse, and treat every item as equally dissimilar.

Embeddings compress them to a dense space where geometry encodes meaning — distances and directions become semantic (the famous *king – man + woman $\approx$ queen*). The model learns these vectors to minimise the task loss, so similar contexts pull items together. They power NLP, recommenders, and any high-cardinality categorical feature.

PROS

- Dense, compact; capture semantic similarity.
- Handle high-cardinality categoricals gracefully.
- Transferable (pre-trained embeddings).

CONS

- Need data to learn well; cold-start for new items.
- Static embeddings ignore context (one vector per word).
- Less interpretable dimensions.

Q: Static (Word2Vec) vs. contextual (BERT) embeddings?

A: Word2Vec/GloVe give one fixed vector per word, so "bank" has the same vector

everywhere. Contextual embeddings from transformers produce a different vector

depending on the surrounding sentence, resolving polysemy — a major reason

transformers replaced static embeddings.

Q: How do you choose embedding dimension?

A: It's a capacity vs. overfitting trade-off tuned by validation. Rules of thumb exist (e.g. ~

the fourth root of cardinality, or 50–300 for words), but I'd sweep a few values. Larger

dims capture more nuance but need more data and compute.

Part 3 — NLP & Transformers

32. Tokenization (BPE / WordPiece) NLP Basics

HOW TO START ANSWERING

"Tokenization is how we split raw text into the units a model actually processes. Modern NLP uses subword tokenization like BPE or WordPiece, which balances vocabulary size against the ability to handle rare and unseen words."

MATH INTUITION

Byte-Pair Encoding (BPE): start from characters, then greedily merge the most frequent adjacent pair, repeating until the vocab reaches a target size.

Intuition: word-level vocabularies are huge and choke on out-of-vocabulary words; character-level is tiny but makes sequences very long. Subwords are the sweet spot — common words stay whole ("the"), rare words break into meaningful pieces ("token" + "ization"). This means no word is ever truly unknown, the vocab stays bounded (~30k–50k), and morphology is captured for free.

PROS

- No OOV problem — rare words decompose.
- Bounded vocabulary; handles morphology.
- Language-agnostic.

CONS

- Tokens $\neq$ words (counting/length quirks).
- Suboptimal for some languages / code / numbers.
- Fixed once trained.

Q: Why not just use words or characters as tokens?

A: Word-level has a massive vocab and fails on unseen words; character-level handles anything but produces very long sequences that are slow and hard to learn long-range patterns from. Subword tokenization gets the best of both — compact vocab, no OOV, reasonable sequence length.

Q: How does tokenization affect cost and context limits?

A: LLM pricing and context windows are measured in tokens, not words (~0.75 words per token in English). Inefficient tokenization (e.g. for code or non-English text) uses more tokens, raising cost and consuming the context window faster.

33. Word Embeddings (Word2Vec / GloVe) NLP Basics

HOW TO START ANSWERING

"Word2Vec and GloVe are classic methods to learn dense word vectors from large text corpora, based on the distributional hypothesis: words appearing in similar contexts have similar meanings. They were the breakthrough that gave words a meaningful geometry."

MATH INTUITION

Word2Vec (Skip-gram): predict surrounding context words from a target word, maximising $\sum \log P(\text{context} | \text{word})$ via a shallow network.

Intuition: "you shall know a word by the company it keeps." By training to predict neighbours, words used in similar contexts get pulled to similar vectors. GloVe instead factorises a global co-occurrence count matrix to the same end. The resulting space has linear structure — analogies like *king* — *man* + *woman* $\approx$ *queen* — because consistent semantic relationships become consistent vector offsets.

PROS

- Capture semantic & syntactic similarity.
- Pre-trained, transferable, cheap to use.
- Dense and compact vs. one-hot.

CONS

- One vector per word — ignores context (polysemy).
- Can encode social biases from corpora.
- No handling of OOV (without subwords).

Q: CBOW vs. Skip-gram?

A: CBOW predicts the target word from its context (faster, better for frequent words);

Skip-gram predicts context from the target (slower but better for rare words and small corpora). Skip-gram with negative sampling is the popular choice.

Q: What's the key limitation that transformers fixed?

A: These embeddings are static — "apple" the fruit and "Apple" the company share one vector. Contextual models (ELMo, BERT) produce vectors conditioned on the whole sentence, resolving polysemy and dramatically improving downstream performance.

34. Self-Attention Transformers

HOW TO START ANSWERING

"Self-attention is the core mechanism of the transformer. It lets every token look at every other token in the sequence and decide which ones are relevant, computing a weighted blend of their information. This is how the model builds context-aware representations."

MATH INTUITION

$$\text{Attention}(Q,K,V) = \text{softmax}(QK^T / \sqrt{d_k}) V$$

Intuition: each token emits a **Query** ("what am I looking for?"), a **Key** ("what do I offer?"), and a **Value** ("what I'll pass on"). The dot product of a query with every key scores relevance; softmax turns scores into weights; we then take a weighted sum of values. So each token's new representation is a context-aware mix of the whole sequence. Dividing by $\sqrt{d_k}$ keeps the dot products from growing too large and saturating the softmax.

PROS

- Captures long-range dependencies directly ($O(1)$ path).
- Fully parallelisable over the sequence.
- Dynamic, content-based weighting.

CONS

- $O(n^2)$ compute/memory in sequence length.
- No inherent notion of order (needs positional encoding).
- Data- and compute-hungry.

Q: Why divide by $\sqrt{d_k}$?

A: For large key dimensions, the dot products have large variance, pushing softmax into regions with tiny gradients (saturation). Scaling by $\sqrt{d_k}$ normalises the variance back to ~ 1 , keeping gradients healthy and training stable.

Q: What is the computational bottleneck and how is it addressed?

A: The QK^T matrix is $n \times n$, so cost grows quadratically with sequence length — the limit on context windows. Efficient variants (FlashAttention for IO-efficiency, sparse/linear attention, sliding-window) reduce the memory or compute to make long contexts feasible.

35. Multi-Head Attention & Positional Encoding Transformers

HOW TO START ANSWERING

"Multi-head attention runs several attention operations in parallel, each in a different learned subspace, so the model can attend to different kinds of relationships at once. Positional encoding is what injects word-order information, since attention itself is order-agnostic."

MATH INTUITION

Multi-head: project Q,K,V into h subspaces, attend independently, concatenate, and project back. **Positional encoding** adds position-dependent signals (sinusoids or learned vectors) to the token embeddings.

Intuition: one attention head can only average one type of relationship; multiple heads let the model simultaneously track syntax, coreference, long-range topic, etc. — like having several "perspectives." Since attention treats input as a set (permuting tokens gives the same result), we must explicitly add order — positional encodings give each position a unique signature so "dog bites man" differs from "man bites dog."

PROS

- Heads capture diverse relationships in parallel.
- Richer representations than single attention.
- Positional encoding restores order cheaply.

CONS

- More parameters/compute (multiple heads).
- Some heads are redundant (prunable).
- Absolute encodings extrapolate poorly to longer lengths.

Q: Why multiple heads instead of one big attention?

A: Different heads specialise — some track local syntax, others long-range dependencies

or specific patterns. Splitting into subspaces lets the model represent several relationship

types at once, which empirically beats a single attention of equal size.

Q: Sinusoidal vs. learned vs. rotary (RoPE) positional encodings?

A: Sinusoidal is fixed and can extrapolate somewhat; learned is flexible but capped at

trained length. RoPE (rotary) encodes relative position by rotating Q/K, generalises better

to longer contexts, and is now standard in modern LLMs like LLaMA.

36. The Transformer Architecture Transformers

HOW TO START ANSWERING

"The Transformer, from 'Attention Is All You Need', replaced recurrence entirely with self-attention. Its big win is parallelism and modelling long-range dependencies directly, which is why it scales and underpins virtually all modern LLMs."

MATH INTUITION

Each block = Multi-Head Attention → Add & LayerNorm → Feed-Forward → Add & LayerNorm, with residual connections throughout.

Intuition: attention mixes information across tokens; the position-wise feed-forward network then processes each token's enriched representation. Residual connections give gradients a highway (enabling deep stacks), and LayerNorm stabilises training. Because there's no recurrence, all positions are computed in parallel — the key to training on massive data. Encoder-decoder (translation), encoder-only (BERT), and decoder-only (GPT) are the three flavours.

PROS

- Massively parallel; scales to huge models/data.
- Captures long-range context directly.
- Transfer-learning friendly (pretrain → fine-tune).

CONS

- Quadratic attention cost limits context.
- Very data- and compute-intensive.
- Large memory footprint; hard to deploy.

Q: Encoder-only vs. decoder-only vs. encoder-decoder — when each?

A: Encoder-only (BERT) sees the whole input bidirectionally — best for understanding

tasks (classification, NER). Decoder-only (GPT) is causal/autoregressive — best for

generation. Encoder-decoder (T5, original Transformer) suits seq2seq like translation and

summarisation.

Q: Why did Transformers replace RNNs?

A: RNNs process tokens sequentially (no parallelism) and struggle with long dependencies

despite gating. Transformers compute all positions in parallel and give every token a

direct path to every other, so they train far faster on more data and capture longer

context — enabling the scaling that produced modern LLMs.

37. BERT vs. GPT Pretrained Models

HOW TO START ANSWERING

"BERT and GPT are both transformer-based but differ in direction and objective. BERT is a bidirectional encoder trained to fill in blanks — great for understanding. GPT is a unidirectional decoder trained to predict the next token — great for generation."

MATH INTUITION

BERT: Masked Language Modelling — randomly mask ~15% of tokens and predict them using both left and right context.

GPT: Causal LM — predict the next token given only previous tokens: $P(x_t | x_1 \dots x_{t-1})$

Intuition: BERT can peek both directions because it's not generating — it builds rich representations for classification/extraction. GPT must be causal (mask future tokens) so it can generate text autoregressively at inference. That difference in objective is why one excels at understanding and the other at generation.

BERT (ENCODER)

- Bidirectional context; strong for NLU.
- Classification, NER, QA, embeddings.
- Fine-tune with a small head.

GPT (DECODER)

- Autoregressive; natural text generation.
- Few-shot / in-context learning.
- Scales into general-purpose LLMs.

Q: Why can't BERT generate text well?

A: BERT is trained to fill masked positions using both sides of context, not to produce

tokens left-to-right. It has no causal structure for autoregressive generation, so it's used

for understanding/embedding tasks, while GPT-style causal models handle generation.

Q: What is the [CLS] token in BERT?

A: A special token prepended to every input; its final hidden state is used as an aggregate

sentence representation for classification tasks. You attach a classifier head on top of the

[CLS] embedding and fine-tune.

38. Sequence-to-Sequence & Encoder-Decoder Attention NLP Tasks

HOW TO START ANSWERING

"Seq2seq models map an input sequence to an output sequence of possibly different length — translation, summarisation, question answering. An encoder reads the input into a representation and a decoder generates the output, with cross-attention letting the decoder focus on relevant input parts."

MATH INTUITION

The decoder generates token by token, at each step attending over the encoder outputs (cross-attention) plus its own past tokens (self-attention).

Intuition: early seq2seq RNNs squeezed the whole input into one fixed vector — a bottleneck that lost detail on long inputs. **Attention** fixed this: instead of one summary, the decoder dynamically looks back at all encoder states and weights the ones relevant to the word it's currently producing (e.g. aligning a French word to its English source). This alignment is exactly what made neural machine translation work.

PROS

- Handles variable input/output lengths.

- Cross-attention removes the fixed-vector bottleneck.
- General framework (translation, summarisation, QA).

CONS

- Autoregressive decoding is sequential/slow.
- Exposure bias (train vs. inference mismatch).
- Can hallucinate or repeat without good decoding.

Q: Greedy vs. beam search vs. sampling for decoding?

A: Greedy picks the top token each step (fast, can be myopic). Beam search keeps the top-k partial sequences for higher-quality, more likely outputs (good for translation).

Sampling with temperature / top-p adds diversity and creativity (good for open-ended generation). Choice depends on whether you want accuracy or variety.

Q: What is teacher forcing and its downside?

A: During training the decoder is fed the true previous token rather than its own

prediction, which speeds and stabilises learning. The downside is exposure bias: at

inference it must consume its own (sometimes wrong) outputs, a distribution it never saw

in training, which can compound errors.

Part 4 — LLMs & Generative AI

39. LLM Training Pipeline (Pretraining → SFT → RLHF) LLM Core

HOW TO START ANSWERING

"Modern LLMs are built in stages: large-scale self-supervised pretraining gives broad knowledge, supervised fine-tuning teaches it to follow instructions, and alignment (RLHF/DPO) makes it helpful and safe. Each stage shapes a different capability."

MATH INTUITION

Pretraining: next-token prediction over trillions of tokens — maximise $\sum \log P(x_t | x_{<t})$.

RLHF: train a reward model on human preference pairs, then optimise the LLM (via PPO) to maximise that reward with a KL penalty to stay near the original model.

Intuition: pretraining is where almost all knowledge is learned — the model compresses the internet by predicting the next token. But a raw pretrained model just continues text; SFT on instruction-response pairs teaches the chat format. RLHF then nudges it toward responses humans prefer, with the KL term preventing it from drifting into degenerate outputs.

WHAT EACH STAGE GIVES

- Pretraining: knowledge & fluency.
- SFT: instruction following, format.
- RLHF/DPO: helpfulness, safety, tone.

CHALLENGES

- Pretraining: enormous compute/data cost.
- RLHF: reward hacking, costly human labels.
- Alignment tax: can reduce raw capability.

Q: What is DPO and why is it popular vs. RLHF/PPO?

A: Direct Preference Optimization reframes alignment as a simple classification loss

directly on preference pairs, skipping the separate reward model and unstable RL (PPO)

loop. It's simpler, cheaper, and more stable while achieving comparable alignment —

hence its rapid adoption.

Q: Where does an LLM's knowledge actually come from?

A: Almost entirely from pretraining — that's where it ingests the world's text. SFT and

RLHF mostly change behaviour and style, not knowledge. That's why you can't teach new

facts reliably with a little fine-tuning; you use retrieval (RAG) instead.

40. Prompt Engineering & In-Context Learning LLM Core

HOW TO START ANSWERING

"Prompt engineering is designing the input to steer an LLM toward the output you want, without changing its weights. It works because of in-context learning — the model adapts to examples and instructions given in the prompt at inference time."

MATH INTUITION

Techniques: zero-shot, few-shot (examples in the prompt), and Chain-of-Thought ("think step by step").

Intuition: the model conditions its next-token distribution on everything in the context, so examples and instructions reshape that distribution toward the desired behaviour — no gradient updates needed. Chain-of-Thought helps because generating intermediate reasoning tokens gives the model more "compute steps" to work through multi-step problems, rather than forcing an answer in one jump. Structure, clear instructions, and good examples dramatically change quality.

PROS

- No training; instant iteration.
- Few-shot adapts to new tasks cheaply.

- CoT boosts reasoning/accuracy.

CONS

- Brittle — small wording changes shift output.
- Consumes context window (cost/limits).
- Not a substitute for real knowledge (still hallucinates).

Q: Why does Chain-of-Thought improve performance?

A: It lets the model externalise intermediate steps, effectively spreading a hard

computation across more token-generation steps instead of demanding the answer

immediately. This especially helps arithmetic, logic, and multi-hop reasoning — though it adds tokens/latency.

Q: Few-shot vs. fine-tuning — when each?

A: Few-shot when you need flexibility, have few examples, or want zero training cost.

Fine-tuning when you have many examples, need consistent format/behaviour at scale,

want to save prompt tokens, or need to bake in a style the model resists. Often start with prompting, graduate to fine-tuning if needed.

41. Retrieval-Augmented Generation (RAG) LLM Systems

HOW TO START ANSWERING

"RAG grounds an LLM in external, up-to-date knowledge by retrieving relevant documents and injecting them into the prompt before generation. It's the standard fix for hallucination and stale knowledge, and it lets the model use private/domain data it was never trained on."

MATH INTUITION

■ Pipeline: embed the query → vector-search a document store (cosine similarity) → retrieve top-k chunks → prepend them to the prompt → the LLM answers grounded in that context.

■ **Intuition:** instead of relying on parametric memory (knowledge frozen in weights), we give the model a "open-book exam" — fetch the relevant passages and let it read them. Retrieval uses embeddings so semantically similar text is found even without keyword overlap. This separates knowledge (easily updated in the database) from reasoning (the LLM), and lets the model cite sources.

PROS

- Reduces hallucination; grounds answers.
- Fresh & private knowledge without retraining.
- Citable, auditable sources.

CONS

- Quality capped by retrieval quality ("garbage in").
- Extra latency & infra (vector DB, chunking).
- Context-window limits on retrieved text.

Q: RAG vs. fine-tuning for adding knowledge?

A: RAG for knowledge that's large, changes often, or needs citations — you update a database, not weights. Fine-tuning for teaching *behaviour*, style, or format, not facts.

They're complementary: fine-tune for how to respond, RAG for what facts to use. RAG is usually the right first choice for factual grounding.

Q: How do you improve a RAG system that gives wrong answers?

A: Most problems are retrieval, not generation. Improve chunking strategy, use better

embedding models, add re-ranking (cross-encoder) over the top results, hybrid

keyword+vector search, query rewriting, and metadata filtering. Then check the prompt

instructs the model to answer only from context and say "I don't know" otherwise.

42. Fine-Tuning & PEFT (LoRA / QLoRA) LLM Systems

HOW TO START ANSWERING

"Full fine-tuning of an LLM updates all billions of parameters — expensive and storage-heavy. Parameter-Efficient Fine-Tuning, especially LoRA, freezes the base model and trains a tiny set of new weights, achieving similar quality at a fraction of the cost."

MATH INTUITION

LoRA: freeze W ; learn a low-rank update $\Delta W = B \cdot A$ where A, B are small (rank r), so $W' = W + BA$. Only A and B are trained.

Intuition: the weight update needed to adapt a model to a task is empirically low-rank — it lives in a small subspace. So instead of updating a huge $d \times d$ matrix, we learn two skinny matrices whose product approximates the update, cutting trainable parameters by $\sim 1000\times$. QLoRA goes further: quantise the frozen base to 4-bit, so a large model fine-tunes on a single GPU. At inference, BA can be merged into W for zero added latency.

PROS

- Tiny memory/compute; train on modest GPUs.
- Swap small adapters per task; base shared.
- Matches full fine-tuning quality often.

CONS

- Slightly below full FT on hard tasks.
- Rank r is a hyperparameter to tune.
- Still needs quality labelled data.

Q: Why does LoRA work — what's the key assumption?

A: That the change in weights for adapting to a downstream task has low intrinsic rank.

Since the meaningful update lives in a small subspace, a low-rank factorisation captures it well, so we get most of the benefit with a fraction of the parameters.

Q: What does QLoRA add over LoRA?

A: It quantises the frozen base model to 4-bit (NF4) and back-propagates LoRA adapters through it, plus tricks like double quantization and paged optimizers. This slashes memory enough to fine-tune 30B-65B models on a single consumer/pro GPU with negligible quality loss.

43. Quantization & Model Compression Deployment

HOW TO START ANSWERING

"Quantization shrinks a model by storing its weights in lower precision — e.g. 16-bit or 8-bit or 4-bit instead of 32-bit floats. It cuts memory and speeds inference, making large models deployable, with minimal accuracy loss when done well."

MATH INTUITION

Map a float range to integers: $x_q = \text{round}(x / \text{scale}) + \text{zero_point}$, then dequantize when computing.

Intuition: neural nets are over-parameterised and robust to small numerical noise, so we don't need full 32-bit precision to represent weights. Reducing precision quarters or eighths the memory and lets hardware do faster integer math. Post-Training Quantization (PTQ) is quick (no retraining); Quantization-Aware Training (QAT) simulates quantization during training for better accuracy at very low bit-widths. Related: pruning (remove weights) and distillation (train a small model to mimic a big one).

PROS

- 4x-8x smaller; faster inference.
- Enables on-device / single-GPU serving.

- Lower cost and energy.

CONS

- Some accuracy loss (worse at very low bits).
- Hardware/kernel support needed for speedups.
- Outlier activations can break naive quantization.

Q: PTQ vs. QAT?

A: Post-Training Quantization is applied to an already-trained model — fast and data-light, good down to 8-bit. Quantization-Aware Training simulates quantization during training so the model learns to be robust to it — needed for aggressive low-bit (4-bit and below) without large accuracy drops, but costs a training run.

Q: Quantization vs. distillation vs. pruning?

A: Quantization lowers numeric precision; pruning removes unimportant weights/neurons for sparsity; distillation trains a smaller "student" to match a larger "teacher's" outputs. They're complementary and often combined to hit a target latency/size budget.

44. Hallucination & LLM Evaluation LLM Core

HOW TO START ANSWERING

"Hallucination is when an LLM produces fluent but false or unsupported content. It happens because the model is trained to predict plausible text, not to verify truth. I'd discuss why it occurs, how to reduce it, and how to evaluate generative models, which is genuinely hard."

MATH INTUITION

The model samples from $P(\text{next token} \mid \text{context})$ — it optimises likelihood/plausibility, with no built-in notion of factual grounding.

Intuition: the objective rewards text that *looks* right, so when the model lacks knowledge it confidently interpolates a plausible-sounding answer. Higher temperature increases this. Mitigations: grounding via RAG, lowering temperature, asking for citations, prompting it to say "I don't know," and verification/self-consistency. Evaluation can't use a single accuracy number — we use benchmarks (MMLU), human/LLM-as-judge ratings, and task-specific metrics.

MITIGATIONS

- RAG grounding + citations.
- Lower temperature; constrained decoding.
- Self-consistency / verification passes.

EVAL CHALLENGES

- No single ground-truth for open generation.
- Benchmarks leak into training data.
- Human eval is costly; LLM-judges are biased.

Q: How do you evaluate a generative LLM in production?

A: A mix: offline benchmarks for capability, task-specific metrics (e.g.

faithfulness/groundedness for RAG), LLM-as-a-judge for scale with human spot-checks for

calibration, plus online signals (user thumbs, A/B tests, refusal/error rates). No single

metric suffices — you triangulate.

Q: Can you eliminate hallucination entirely?

A: Not fully with current architectures — it's inherent to probabilistic next-token

generation. You reduce it substantially with grounding (RAG), guardrails, citations, and

verification, and you design the product to handle residual errors (show sources, allow

"I'm not sure," human-in-the-loop for high stakes).

45. LLM Agents & Tool Use GenAI Systems

HOW TO START ANSWERING

"An LLM agent uses the model as a reasoning engine that can plan, call external tools, observe results, and iterate toward a goal — rather than answering in one shot. Tools (search, code, APIs) give it abilities and current information the base model lacks."

MATH INTUITION

The **ReAct** loop: Thought → Action (call a tool) → Observation → repeat → Final Answer.

Intuition: the LLM can't browse, compute exactly, or know today's data, but it *can* decide which tool to call and interpret the result. By interleaving reasoning ("Thought") with tool calls ("Action") and feeding outputs back in ("Observation"), it decomposes complex tasks and grounds itself in real results — turning a static text predictor into something that acts. Memory and planning modules extend this to long-horizon tasks.

PROS

- Overcomes knowledge/compute limits via tools.
- Handles multi-step, dynamic tasks.
- Grounded, up-to-date, verifiable actions.

CONS

- Error compounding over many steps.
- Latency, cost, and reliability concerns.
- Safety: tool access can cause real-world side effects.

Q: Why do multi-step agents become unreliable?

A: Errors compound — if each step is 90% reliable, ten steps drop to ~35%. A single bad tool call or misread observation can derail the whole chain. Mitigations: constrain the action space, add verification/reflection steps, allow retries, and keep humans in the loop for critical actions.

Q: How does function/tool calling actually work?

A: The model is given tool schemas; instead of free text it emits a structured call (name + JSON arguments) when it decides a tool is needed. Your system executes the tool, returns the result into the context, and the model continues. Fine-tuning/training on this format makes the calls reliable and parseable.

46. Generative Models: GANs, VAEs & Diffusion GenAI

HOW TO START ANSWERING

"These are the main families of generative models for images and other continuous data. I'd frame them by how they learn the data distribution: VAEs use a probabilistic latent space, GANs use an adversarial game, and diffusion models learn to denoise — with diffusion now dominant for high-quality image generation."

MATH INTUITION

GAN: a generator and discriminator play a minimax game — G tries to fool D, D tries to spot fakes.

Diffusion: gradually add Gaussian noise to data (forward), then train a network to reverse it step by step (denoise) to generate samples.

Intuition: a VAE compresses data to a latent distribution and decodes samples (smooth latent space but blurry outputs). A GAN learns by competition — sharp results but unstable training and mode collapse. Diffusion turns generation into many small, stable denoising steps, which is easier to train and produces state-of-the-art, diverse, high-fidelity images — at the cost of slow multi-step sampling.

DIFFUSION PROS

- State-of-the-art quality & diversity.
- Stable training (vs. GANs).
- Controllable (text-conditioning, guidance).

DIFFUSION CONS

- Slow sampling (many denoising steps).
- Compute-intensive.
- Large models; harder to deploy.

Q: What is mode collapse in GANs?

A: When the generator finds a few outputs that reliably fool the discriminator and

produces only those, ignoring the diversity of the real data. The result is realistic but

repetitive samples. It's a key reason diffusion models, which don't suffer this, overtook

GANs for image generation.

Q: Why did diffusion overtake GANs?

A: Diffusion reframes generation as a sequence of easy denoising steps with a stable,

likelihood-style objective — avoiding GANs' fragile adversarial dynamics and mode

collapse. It yields higher quality and diversity and conditions cleanly on text, enabling

systems like Stable Diffusion. The trade-off is slower, multi-step sampling.